

A High Efficiency Hardware Design for the Post-Quantum KEM HQC

Francesco Antognazza, Alessandro Barenghi, Gerardo Pelosi

Dept. of Electronics, Information and Bioengineering - DEIB, Politecnico di Milano, Milano, Italy

{francesco.antognazza, alessandro.barenghi, gerardo.pelosi}@polimi.it

Ruggero Susella

STMicroelectronics S.r.l., Agrate Brianza (MB), Italy

ruggero.susella@st.com

Abstract—In this work, we present the first high-performance yet compact RTL design of the HQC cryptographic algorithm, fully compatible with the last specification revision of April 2023, which added implicit rejection calling. Our design improves on the state of the art of the specialized polynomial multipliers and Reed-Solomon/Reed-Muller decoder components, which take the largest share of the HQC computation time. Furthermore, we compare the efficiency of the sparse polynomial sampler proposed by the HQC team with different approaches proposed by the research community. We benchmarked our design employing the Xilinx Artix-7 FPGA line, selected by the US NIST as the reference benchmarking platform for post-quantum cipher implementation. We report improvements in the latency for keygen, encapsulation, and decapsulation operations between $1.58\times$ and $2.93\times$, and efficiency improvements (in terms of execution time per hardware resources) from $1.23\times$ to $1.62\times$, with respect to the current state of the art RTL implementations of HQC components. When compared with the specification compliant HLS implementation provided by the HQC team, we achieve speedups between $7.41\times$ and $10.67\times$, and improve the overall efficiency by a factor between $6.05\times$ and $10.98\times$.

Index Terms—Post-quantum Cryptosystems, Code-based Cryptosystems, Hardware Security

I. INTRODUCTION

Post-quantum cryptosystems have been the subject of a significant amount of research, both in terms of their mathematical design and concrete realization. This interest was driven by the large amount of effort put into the engineering of large scale quantum computers which, besides providing substantial speedups in useful computations, would also break well established asymmetric cryptographic primitives relying on the hardness of the discrete logarithm problem and the integer factorization problem. To this end, the US National Institute of Standards and Technology (NIST) started the Post-Quantum Cryptography (PQC) standardization process, aiming to assess and select the most promising asymmetric cryptographic algorithms based on different classes of computationally hard problems not easily solvable even by quantum computers. Through two rounds of this evaluation process, NIST selected seven finalists and eight alternate candidates, which were then thinned down to two standard candidates and four alternates for a final round and possible standardization. Notably NTRU [1], [2], one among the said finalists, was adopted by Google to secure its internal cloud communications [3], [4]. The current NIST standardization-elected

candidate Key Encapsulation Method (KEM) is Kyber [5], a lattice-based cryptosystem, while selected four alternates relying on different computationally hard problems to obtain resilience against potentially disruptive attacks on lattice-based cryptosystems. Nonetheless, a polynomial-time cryptanalytic attack eliminated SIKE, leaving only three KEMs that are based on the hardness of the syndrome decoding problem, i.e., Classic McEliece [6], BIKE [7] and HQC [8]. Among these, only HQC exhibits a security proof based on a polynomial reduction of the solution of a computationally hard problem to the breaking of the cryptosystem. In particular, any successful break of HQC implies a faster way to solve the syndrome decoding problem on a random Quasi-Cyclic (QC) code of rate $\frac{1}{2}$. By contrast, Classic McEliece relies on an additional assumption, namely the indistinguishability of a hidden Goppa code from a random code, while BIKE relies on the indistinguishability of a hidden QC-MDPC code from a random QC code (no proof that link them to computationally hard problems is known). Hamming Quasi-Cyclic (HQC) [8] is a code-based IND-CCA secure KEM whose security relies on the syndrome decoding problem for QC codes. It is obtained by applying the Hofheinz-Hövelmanns-Kiltz (HHK) transform [9] to an IND-CPA public-key encryption scheme, which is denoted as HQC.PKE. Only in recent times HQC received some research effort on efficient hardware implementations, particularly to accelerate the polynomial multiplication operation [10] or the sample of random polynomials [11]. For the complete design of the algorithm updated to the specification submitted for the third selection round, apart from the early designs resulting from the High Level Synthesis (HLS) provided by the HQC authors [12], the first design described at Register Transfer Level (RTL) is presented in [13]. Moreover, the hardware/software co-design in [14] provides a RISC-V instruction set extension to accelerate various algorithms composing the cryptographic scheme.

Contributions. We present the first constant-time hardware design at the Register Transfer Level (RTL) of the HQC KEM, fully compliant with the latest specification of April 30th, 2023. In doing so, our contributions are the following: *i)* we provide the first, fully detailed design, of the Reed-Solomon/Reed-Muller concatenated code encoder and decoder, the most demanding operation in HQC.PKE de-

encryption, tailored to the cryptographic-size code parameters required in HQC, *ii*) we design a flexible architecture for the specific binary polynomial arithmetic, *iii*) we report a new approach implementing the critical modulo operation in the hardware design of the sampling algorithm for sparse polynomials introduced in the latest revision of the official HQC specification [8] that does not require Digital Signal Processing (DSP) units, and compare it with other techniques proposed by the research community, and *iv*) we propose a simple modification to the HQC specification which, without any security loss, improves the latency in our designs across all primitives between 32% (hqc128) and 13% (hqc256), but software implementations would also gain some benefits.

Our design is intended for deployment on an ASIC target, however, to provide a fair efficiency comparison to the current state of the art, we report the results of our benchmarks on the Xilinx Artix-7 FPGA series, which was chosen by NIST as the reference platform for hardware development. We achieve a speedup between $1.58\times$ and $2.93\times$ with respect to the fastest RTL solution in the current state of the art, and between $7.41\times$ and $10.67\times$ when compared with the HLS realization proposed by the HQC team. Concerning efficiency in terms of area-time product, we obtain gains in the $1.23\times$ to $1.62\times$ range against the current RTL implementations, and between $6.05\times$ and $10.98\times$ against the current HLS implementations.

II. BACKGROUND

The main characteristic of HQC is the use of two linear codes, a (quasi-cyclic) random one providing the security of the scheme, and a fixed public one with an efficient decoding algorithm and high correction capacity. HQC uses the public code to encode a small random string, from which the shared session key is derived, and employ the secret key to (intentionally) corrupt the resulting codeword way beyond the error-correcting capability of the public code. Only the agent who has generated the cryptographic key pair can reduce the corruption and retrieve the original message. In the following, we detail the structure of the HQC.KEM, starting from the underlying HQC.PKE primitives, and describing how the HHK transformation [9] is applied to it to obtain the full KEM.

Let $\mathbb{F}_2$ be the binary finite field, and $\mathbf{R}$ be the polynomial ring $\mathbb{F}_2[x]/\langle x^p - 1 \rangle$, where $p > 3$ is a prime number such that $\text{ord}_2(p) = p - 1$. In this way $x^p - 1 \in \mathbb{F}_2[x]$ admits only two irreducible factors (mod p) and mathematical attacks based on the ring structure are thwarted. Given an element $a \in \mathbf{R}$, we denote its Hamming weight (i.e., the number of non-zero binary coefficients) as $\omega(a)$. The set containing all polynomials in $\mathbf{R}$ with Hamming weight $w \geq 1$, is denoted as $\mathbf{R}_w$. We consider a polynomial $a = a_0 + a_1x + \dots + a_{p-1}x^{p-1} \in \mathbf{R}$ as a p -dimensional binary vector composed by its coefficients listed in the little-endian format (i.e., from the constant term a_0 onwards) and denote it with a lowercase bold letter: $\mathbf{a} = [a_0, a_1, \dots, a_{p-1}] \in \mathbb{F}_2^p$. While the addition of two polynomials $a, b \in \mathbf{R}$ corresponds to a component-wise Boolean xor between their vector representations (i.e., $a_i \oplus b_i$, $0 \leq i < p$), it is useful to note that the vector representation

Algorithm 1 HQC.PKE-KEYGEN

Require: None

Ensure: $\text{pk} = (\varphi \in \{0, 1\}^{320}, s \in \mathbf{R})$ $\text{sk} = \sigma \in \{0, 1\}^{320}$

- 1: $(\sigma, \varphi) \xleftarrow{\$} \{0, 1\}^{320} \times \{0, 1\}^{320}$
 - 2: $h \leftarrow \text{CSPRNG}(\varphi, \mathbf{R})$
 - 3: $(x, y) \leftarrow \text{CSPRNG}(\sigma, \mathbf{R}_w \times \mathbf{R}_w)$
 - 4: $\mathbf{s} \leftarrow x + h \cdot y$
 - 5: **return** $\text{pk} = (\varphi, \mathbf{s})$, $\text{sk} = \sigma$
-

Algorithm 2 HQC.PKE-ENCRYPT

Require: $\text{pk} = (\varphi \in \{0, 1\}^{320}, s \in \mathbf{R})$, $\mathbf{m} \in \{0, 1\}^k$, $\theta \in \{0, 1\}^{512}$

Ensure: $\text{ctx} = (u \in \mathbf{R}, \mathbf{v} \in \mathbb{F}_2^{n_e n_i})$

- 1: $(e, r_a, r_b) \leftarrow \text{CSPRNG}(\theta, \mathbf{R}_{w_e} \times \mathbf{R}_{w_r} \times \mathbf{R}_{w_r})$
 - 2: $h \leftarrow \text{CSPRNG}(\varphi, \mathbf{R})$
 - 3: $u \leftarrow r_a + h \cdot r_b$
 - 4: $\mathbf{v} \leftarrow \text{ENCODE}_G(\mathbf{m}) + \text{TRUNC}(s \cdot r_b + e)$
 - 5: **return** $\text{ctx} = (u, \mathbf{v})$
-

of the result of their product $c = a \cdot b$ is obtained as: $c_i = \bigoplus_{j+k \equiv i \pmod p} (a_j \text{ and } b_k)$, with $i, j, k \in \{0, 1, \dots, p-1\}$.

A $[n, k, d]$ binary linear (forward error-correcting) code $\mathcal{C}$ is defined as a subspace of $\mathbb{F}_2^n$ of dimension k , where any two (row) vectors differ by at least d coordinates (thus, d is also known as the minimum distance of $\mathcal{C}$). We denote with $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ the generator matrix of the code, which allows to generate all the 2^k codewords $\mathbf{c} \in \mathcal{C} \subset \mathbb{F}_2^n$ from the information words $\mathbf{m} \in \mathbb{F}_2^k$ as $\mathcal{C} = \{\mathbf{m}\mathbf{G}, \mathbf{m} \in \mathbb{F}_2^k\}$. Similarly, we denote as $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ the parity-check matrix of $\mathcal{C}$ such that $\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_2^n \mid \mathbf{H}\mathbf{c}^T = \mathbf{0}\}$ and denote as syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$ of $\mathbf{v} \in \mathbb{F}_2^n$ with respect to $\mathbf{H}$ the result of $\mathbf{H}\mathbf{v}^T$ for a generic $\mathbf{v} \in \mathbb{F}_2^n$.

HQC uses a random quasi-cyclic $[2p, p, d]$ code with a public parity-check matrix $\mathbf{H} = [\mathbf{I}_p \mid \text{rot}(h)]$. $\mathbf{I}_p$ is the $p \times p$ identity matrix, while $\text{rot}(h)$ is the $p \times p$ matrix obtained from the juxtaposition of the column vector $\mathbf{h}$ representing the binary coefficients of a random polynomial $h \in \mathbf{R}$, and a sequence of columns each of which obtained through a vertical rotation of the previous one by one cell. The public fixed code is generated by the concatenation of a shortened Reed-Solomon (RS) $[n_e, k_e, d_e]$ (external) code with a duplicated Reed-Muller (RM) $[n_i, k_i, d_i]$ (internal) code, thus its length, dimension and minimum distance are $[n_e n_i, k_e k_i, d_e d_i]$. In the following, we denote the publicly known RS-RM generator matrix as $\mathbf{G}$. We denote as $\text{CSPRNG}(\eta, \mathbf{S})$ the procedure to pick an item from the set $\mathbf{S}$ applying a statistically uniform sampling strategy that makes use of a Cryptographically Secure PseudoRandom Number Generator (CSPRNG), which in turn takes as input a seed $\eta \in \{0, 1\}^l$, $l \geq 1$. In particular, in HQC, the SHAKE256 algorithm, from the SHA-3 NIST standard [15], is used to implement the CSPRNG.

A. HQC Public Key Encryption

The key generation procedure of HQC.PKE (Algorithm 1) starts by sampling two random seeds σ and φ of 320 bits each. φ is used to sample the parity-check matrix of the random QC code $\mathbf{H} = [\mathbf{I}_p, \text{rot}(h)]$, using a random polynomial h sampled from $\mathbf{R}$. The seed σ is used to sample the secret polynomials x and y , both with fixed Hamming weight $\omega(x) = \omega(y) =$

Algorithm 3 HQC.PKE-DECRYPT

Require: $\text{sk} = \sigma \in \{0, 1\}^{320}$, $\text{ctx} = (u \in \mathbf{R}, \mathbf{v} \in \mathbb{F}_2^{n_e n_i})$
Ensure: $\mathbf{m}' \in \{0, 1\}^k$
1: $(x, y) \leftarrow \text{CSPRNG}(\sigma, \mathbf{R}_w \times \mathbf{R}_w)$
2: $\mathbf{m}' \leftarrow \text{DECODE}_G(\text{TRUNC}([\mathbf{v} \| 0^{p-n_e n_i}] - u \cdot y))$
3: **return** $\mathbf{m}'$

w . The outputs of the procedure are the the private key seed σ and the public key value (φ, s) , where s is the syndrome polynomial of the pair (x, y) in the random quasi-cyclic code defined by $\mathbf{H}$ (stored as the seed φ of the CS RNG which generates it), $s \leftarrow x + h \cdot y \Leftrightarrow \mathbf{s} = \mathbf{H}[\mathbf{x} \mid \mathbf{y}]^\top$.

The HQC.PKE encryption procedure (Algorithm 2) receives as input the public key (φ, s) , the random message $\mathbf{m}$ to be transmitted, encoded with $k \in \{128, 192, 256\}$ bits, and a 512-bit public salt θ employed to randomize the ciphertext and achieve the IND-CPA property. Three random polynomials $e, r_a, r_b \in \mathbf{R}$, are randomly sampled using the seed θ such that $\omega(e) = w_e$ and $\omega(r_a) = \omega(r_b) = w_r$. The parity-check matrix $\mathbf{H}$ is regenerated from the public seed φ and used to compute the syndrome polynomial u of the bit vector $[\mathbf{r}_a \mid \mathbf{r}_b]$ as $u \leftarrow r_a + h \cdot r_b \Leftrightarrow \mathbf{u} = \mathbf{H}[\mathbf{r}_a \mid \mathbf{r}_b]^\top$. The codeword resulting from the encoding of the message $\mathbf{m}$ with the public RS-RM code with generator matrix $\mathbf{G}$ is then corrupted adding the first $n_e n_i$ bits of the p bits obtained from the $s \cdot r_b + e$ operation ($p > n_e n_i$). The resulting vector $\mathbf{v} \in \mathbb{F}_2^{n_e n_i}$ cannot be used to obtain $\mathbf{m}$ using the public RS-RM decoding algorithm alone, since we added far more error bits than its maximum correcting capability. The ciphertext is defined as $(u, \mathbf{v})$.

The HQC.PKE decryption algorithm (Algorithm 3) starts by expanding the secret vector $[\mathbf{x} \mid \mathbf{y}] \in \mathbb{F}_2^{2p}$ from the secret key $\text{sk} = \sigma$ and subtracts from the polynomial derived from the second component $\mathbf{v}$ of the ciphertext, v , the product of the first one u , by the second element of the secret vector y . The resulting quantity, $v - u \cdot y$ can be shown to be close to a codeword of the RS-RM public code $\mathbf{G}$ as $v - u \cdot y = \mathbf{m}\mathbf{G} + (s \cdot r_b + e - h \cdot r_b \cdot y - r_a \cdot y)$, and, since $h \cdot y = s - x$ from the keygen algorithm, we have: $\mathbf{m}\mathbf{G} + (s \cdot r_b + e - s \cdot r_b + x \cdot r_b - r_a \cdot y) = \mathbf{m}\mathbf{G} + (e + x \cdot r_b - r_a \cdot y)$. Subsequently, the errors added to the codeword $\mathbf{m}\mathbf{G}$ are removed by applying the RS-RM decoding algorithm for the given generation matrix $\mathbf{G}$, and the original message is derived. The HQC.PKE parameters (i.e., $p, w, w_e, w_r, n_i, n_e, k_i, k_e, d_i, d_e$, with $k = k_i k_e$) are tuned so that this decoding action has a negligible failure rate. In the call for post-quantum cryptographic schemes, NIST provided a security level classification to categorize each cipher. Specifically, security levels 1, 3, and 5 correspond to the lowest computational efforts needed to derive the secret key of AES-128, AES-192, AES-256 via the best classical and quantum cryptanalytic attacks, respectively. The designers of HQC provided parameters for the cryptosystem, reported in table Table I, so that the decryption failures take place with a probability of $2^{-\lambda}$, where λ is the bit-length of the key of the AES cipher considered at the corresponding security level.

TABLE I: HQC parameter sets. Public fixed codes are specified via the $[n, k, d]$ notation. The rightmost column reports the Decoding Failure Rate (DFR) of each parameter set.

Sec. level	Param. set	Public concatenated code		Polynomial ring $\mathbf{R}$			DFR
		Reed-Solomon	Reed-Muller	p	w	$w_e = w_r$	
1	hqc128	[46, 16, 31]	[384, 8, 192]	17669	66	75	2^{-128}
3	hqc192	[56, 24, 33]	[640, 8, 320]	35851	100	114	2^{-192}
5	hqc256	[90, 32, 59]	[640, 8, 320]	57637	131	149	2^{-256}

Algorithm 4 HQC.KEM-ENCAPSULATE

Require: $\text{pk} = (\varphi \in \{0, 1\}^{320}, s \in \mathbf{R})$
Ensure: $\text{ctx} = (u \in \mathbf{R}, \mathbf{v} \in \mathbb{F}_2^{n_e n_i}, \text{salt} \in \{0, 1\}^{128}), K \in \{0, 1\}^{512}$
1: $\mathbf{m} \xleftarrow{\$} \{0, 1\}^k$, $\text{salt} \xleftarrow{\$} \{0, 1\}^{128}$
2: $\theta \leftarrow \text{HASH-G}(\mathbf{m} \| \varphi \| \text{salt})$
3: $(u, \mathbf{v}) \leftarrow \text{HQC.PKE-ENCRYPT}(\text{pk}, \mathbf{m}, \theta)$
4: $K \leftarrow \text{HASH-K}(\mathbf{m} \| u \| \mathbf{v})$
5: **return** $\text{ctx} = (u, \mathbf{v}, \text{salt}), K$

Algorithm 5 HQC.KEM-DECAPSULATE

Require: $\text{ctx} = (u \in \mathbf{R}, \mathbf{v} \in \mathbb{F}_2^{n_e n_i}, \text{salt} \in \{0, 1\}^{128}), \text{sk} = (\sigma \in \{0, 1\}^{320}, \delta \in \{0, 1\}^k), \text{pk} = (\varphi \in \{0, 1\}^{320}, s \in \mathbf{R})$
Ensure: $K \in \{0, 1\}^{512}$
1: $\mathbf{m}' \leftarrow \text{HQC.PKE-DECRYPT}(\text{sk}, (u, \mathbf{v}))$
2: $\theta' \leftarrow \text{HASH-G}(\mathbf{m}' \| \varphi \| \text{salt})$
3: $\text{ctx}' \leftarrow \text{HQC.PKE-ENCRYPT}(\text{pk}, \mathbf{m}', \theta')$
4: **if** $\text{ctx} \neq \text{ctx}'$ **then**
5: $K' \leftarrow \text{HASH-K}(\delta \| \text{ctx})$
6: **else**
7: $K' \leftarrow \text{HASH-K}(\mathbf{m}' \| \text{ctx})$
8: **end if**
9: **return** K'

B. HQC Key Encapsulation Method

The HQC.KEM is obtained wrapping the HQC.PKE with the HHK transformation [9] which, from a functional standpoint, feeds the HQC.PKE with a random message, from which the secret to be employed as a session key K is derived, and adds to the ciphertext additional information which allows to check if a decryption error took place. In this case, to avoid information leakage, the construction emits a random string, deterministically derived with a CS RNG from the ciphertext and a secret seed stored with the private key. As a consequence, the key generation of HQC.KEM matches the one of HQC.PKE, save for the generation of an additional seed, denoted as δ , of 128, 192, or 256 bits (depending on the security level), stored together with the private key seed σ .

The HQC.KEM encapsulation algorithm (Algorithm 4) starts by picking a uniformly distributed random message $\mathbf{m}$ encoded with 128, 192, or 256 bits, depending on the security level, and a 128-bit public salt. The two quantities are concatenated with the seed value φ of the HQC public key and hashed via the HASH-G function to get a digest θ , which is subsequently employed as the ephemeral value required as an input by the HQC.PKE encryption algorithm. After calling the routine HQC.PKE-ENCRYPT, the shared secret session key K is derived through a different hash function, HASH-K, fed with the concatenation of the message $\mathbf{m}$ and the ciphertext.

The HQC.KEM decapsulation procedure (Algorithm 5) starts by computing the output value $\mathbf{m}'$ of the routine

HQC.PKE-DECRYPT fed with the secret key and the received ciphertext, retrieving the original confidential message $\mathbf{m}$ unless a decryption failure occurs. To distinguish between the two possible scenarios and also provide security against active attackers, which may have mangled the received ciphertext ctx , the HHK transformation followed by the decapsulation procedure mandates to compare the ctx value with the output obtained from the (re-)computation of the routine HQC.PKE-ENCRYPT, which in turn requires also the re-computation of the ephemeral value θ fed to it as last input parameter. If the retrieved secret message $\mathbf{m}'$ matches the one that was actually encrypted, the outcome of such process (lines 2 and 3 of Algorithm 5) will yield a value ctx' matching the actual ciphertext ctx (check performed at line 4). In case the recomputed ciphertext ctx' does not match the received one, the last version of the official HQC specification (Apr. 2023) forces the so-called *implicit* rejection, that is computation of the HASH-K function fed with the concatenation of the received ciphertext and the binary string δ included in the secret key, instead of the confidential message chosen randomly by the party which performed the HQC.KEM encapsulation procedure.

III. HQC HARDWARE DESIGN

In this section, we describe our design to realize the full HQC KEM hardware module. Our RTL design is organized with a Finite State Machine (FSM) orchestrating accesses to memories as well as computations of arithmetic functional units, RS-RM encoders and decoders, Hash functions and CSPRNG modules. We first describe the design of each computational unit/module (see Sub-sections III-A, III-B, III-C), and subsequently provide a description of the optimized schedule followed by our FSM-based controllers (see Sub-section III-D) to compute the HQC.KEM-KEYGEN, -ENCAPSULATE, and -DECAPSULATE primitives, which in turn use the HQC.PKE-KEYGEN, -ENCRYPT, and -DECRYPT primitives (that include the execution of specific CSPRNG algorithms), as well as the HASH-G and HASH-K hash functions. HQC.PKE primitives use polynomial multiplications between elements in $\mathbf{R} = \mathbb{F}_2[x]/(x^p - 1)$. In particular, the algorithm executed by the HQC.PKE-KEYGEN requires one multiplication between elements in $\mathbf{R}$, where the polynomial employed as a second factor has only w non-null coefficients, with $w \ll p$. Thus, we realize the first factor as a *dense polynomial*, i.e., a sequence of p binary coefficients in little-endian format organized as a list of $\lceil \frac{p}{B} \rceil$ words with a configurable bit-length B (e.g., $B \in \{32, 64, 128\}$), and the second one as a *sparse polynomial*, i.e., a sequence of 16-bit unsigned integers each of which is the exponent of the non-null monomial term x^i , $0 \leq i \leq p-1$. The algorithm executed by the HQC.PKE-ENCRYPT requires two multiplications in $\mathbf{R}$ to compute the ciphertext. Each multiplication exhibits the first factor as a dense polynomial, while the second one is a sparse polynomial with a number of coefficients $w_r \ll p$. Finally, the algorithm executed by the HQC.PKE-DECRYPT requires one multiplication in $\mathbf{R}$, where the first factor is a

dense polynomial and the second one is a sparse polynomial with $w \ll p$ non-null coefficients.

A. Arithmetic Units

HQC.KEM requires only additions and multiplications between polynomials in $\mathbf{R}$, as the single required subtraction is realized as an addition due to the arithmetic in $\mathbb{F}_2[x]$.

1) *Polynomial Adder*: We add/subtract two dense polynomials xor -ing them coefficient-wise. As the operands are stored in memories as sequences of B -bit wide words, $\lceil \frac{p}{B} \rceil$ word-wise xors are carried out. HQC.PKE-ENCRYPT requires two additions between a sparse and a dense polynomial, producing a dense result, while HQC.PKE-keygen performs one such operation. We realize such an addition by flipping the bits of the dense operand in the positions indicated by the sparse one. Each index of the sparse polynomial is stored as 16-bits unsigned integer: we thus split it determining the dense operand word address (as the highest $16 - \log_2(B)$ bits of the index encoding), and the position of the bit to be flipped within the word (as the lowest $\log_2(B)$ bits of the index encoding). Since two consecutive indexes may flip bits in the same result word, creating a *read-after-write* data dependency, pipelining the adder circuit would not improve its performance, when memory access latencies are taken into account. We therefore employ a sequential architecture, also motivated by the low weight of the sparse polynomials (see Table I).

2) *Polynomial Multiplier*: The HQC.KEM primitives require only multiplications where one operand is a sparse polynomial and the other is dense one. While sub-quadratic approaches for generic polynomial multiplication exist, we note that, in the HQC case, where the amount of non-null coefficients of one of the factors is $w \approx O(\sqrt{p})$ out of p , the schoolbook approach has an asymptotic complexity of $O(p^{1.5})$, which is already better than the one of the Karatsuba approach. Furthermore, the schoolbook method does not hide large constants within the asymptotic notation, and allows for a resource-sparing implementation. In particular, we adopt a *shift-and-add* approach for our multiplier, where the dense operand is shifted by an amount specified by each index of the sparse operand, and added to the result. We reduce the memory requirements to the minimum possible by immediately reducing modulo $x^p - 1$ the shifted polynomial to be added. This is possible through a simple change in the location where the coefficients of the dense polynomial are added, as the polynomial modular reduction modulo $x^p - 1$ amounts to a bit-wise xor of the coefficients of degrees greater or equal than p onto the coefficients of the monomials of the result having a degree lower by exactly p units. Our word-wise shift-and-add approach rotates and accumulates a B -bit word at a time, in turn taking $w \cdot \lceil \frac{p}{B} \rceil$ clock cycles and using $\lceil \frac{p}{B} \rceil$ memory blocks to store the result. We note that this algorithm runs in constant time as the memories of our RTL design are uncached, thus fully eliminating the timing side channel which would be present in an analogue software implementation. Furthermore, our design of the dense by sparse polynomial multiplier parallelizes the addition of up

to $l = B/16$ coefficients of the sparse polynomial, exploiting the fact that more than one 16-bit value describing it can be stored in a B -bit word of our design. To this end, we employ l read ports accessing the dense binary polynomial operand, a read and write port for the accumulated result, and a read port for the sparse polynomial. We use a pipelined Barrel module to perform the shift operation, and we parametrized it in the size of the memory blocks B and the number of pipeline stages to break the possibly long critical path at our will.

B. Reed-Solomon & Reed-Muller Encoder and Decoder

HQC uses a public concatenated code composed by a shortened Reed-Solomon (RS) as the external code, and a duplicated Reed-Muller (RM) as the internal one. The RS-RM encoding procedure expands each byte, taken as a message symbol of the Reed-Solomon code, into a 384-bit or 640-bit codeword of the Reed-Muller code. While the RM and RS encoders can work in a pipelined fashion, processing the original input message byte-wise, this is not the case for the concatenated RS-RM decoder, since the RS decoder requires all the codeword symbols for a correct processing.

1) *Reed-Solomon Encoder and Decoder:* The HQC official specification reports an instantiation of a $[n_e, k_e, d_e]$ RS code for each NIST security level, all able to correct $t = \frac{d_e-1}{2}$ erroneous codeword symbols: RS-1=[255, 225, 31], RS-2=[255, 223, 33], and RS-3=[255, 197, 59]. The shortened versions of these codes that are actually used in the HQC.KEM primitives are obtained by fixing 209, 199, or 165 symbols of the message, respectively, thus not transmitting them in the codeword and exhibiting RS-S1=[46, 16, 31], RS-S2=[56, 24, 33], and RS-S3=[90, 32, 59]. Each code treats a block of data as a set of $\mathbb{F}_{2^8}$ elements called symbols, represented employing the irreducible polynomial $y^8 + y^4 + y^3 + y^2 + 1 \in \mathbb{F}_2[y]$. In particular, the message to be encoded is used to build a *message polynomial* $u(x) \in \mathbb{F}_{2^8}[x]$, with degree $k_e - 1$. An n_e -symbol codeword $c(x) = u(x)g(x) \in \mathbb{F}_{2^8}[x]$ is obtained multiplying the input message polynomial $u(x) \in \mathbb{F}_{2^8}[x]$ by the code *generator polynomial* $g(x) \in \mathbb{F}_{2^8}[x]$, which has degree $d_e = n_e - k_e + 1$.

The RS code generator polynomial is defined¹ as $g(x) = (x - \alpha) \cdot (x - \alpha^2) \cdot (x - \alpha^3) \cdots (x - \alpha^{d_e-1})$, where α is the primitive element in $\mathbb{F}_{2^8}$ having representation $\alpha = x$. As a consequence, the first d_e powers of α are roots of both $g(x)$ and any error-free codeword $c(x)$. A systematic encoding procedure, in which each codeword is still a multiple of $g(x)$ and contains the sequence of symbols of the message polynomial $u(x)$ as a prefix and the error correcting symbols as a suffix, is obtained computing $c(x) = x^{n_e-k_e}u(x) - (x^{n_e-k_e}u(x) \bmod g(x))$.

a) *Reed-Solomon Encoder:* Figure 1 shows the linear feedback shift register based design to compute $(x^{n_e-k_e}u(x) \bmod g(x))$ in exactly k_e clock cycles [16]. The RS codeword in systematic form is obtained as the sequence of the first k_e symbols produced in output (which coincide with the original message) followed by the symbols read out

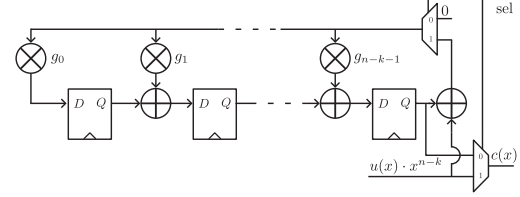


Fig. 1: Circuit computing $u(x) \cdot x^{n_e-k_e} \bmod g(x)$

from the memory elements just after the k_e -th input symbol is consumed. We note that, in the HQC case, the $n_e - k_e$ $\mathbb{F}_{2^8}$ multipliers along the feedback network have a fixed operand value g_i , $0 \leq i \leq n_e - k_e - 1$. Furthermore, each of these fixed operands lies in a small subset of all possible values in $\mathbb{F}_{2^8}$. Since the fully combinatorial network of a generic multiplier for $\mathbb{F}_{2^8}$ would include both **and** and **xor** gates and exhibit the worst critical path for the execution of any multiplication, the previous observations allow us to improve the overall performance of the encoder specializing the Mastrovito bit-parallel design of binary finite fields multipliers [17] for a constant operand and for each distinct value g_i . Indeed, a net advantage in terms of critical path will be in place as there will be no need to employ **and** gates. As multiple $\mathbb{F}_{2^8}$ specialized multipliers will be used simultaneously for the realization of the RS encoder, the absence of **and** gates will reduce the encoder cost in terms of gate count with respect to a generic, yet optimized, $\mathbb{F}_{2^8}$ multiplier. More refined bit-parallel designs, as the ones in [18], have lower performance gains in the case of HQC due to the small size of $\mathbb{F}_{2^8}$.

b) *Reed-Solomon Decoder:* The typical algebraic RS decoder takes as input an error-affected codeword and considers it as a polynomial $r(x) \in \mathbb{F}_{2^8}[x]$, with degree n_e , obtained from the addition of an error-free codeword $c(x) \in \mathbb{F}_{2^8}[x]$, with degree n_e , to an unknown error polynomial $e(x) \in \mathbb{F}_{2^8}[x]$, with up to t terms. The decoder computes the polynomial associated to the syndrome of the received word $r(x)$ and derives both positions and values of the coefficients of $e(x)$. Finally, the error-free codeword is derived as $c(x) = r(x) - e(x)$. The coefficients of the syndrome polynomial $S(x) = S_0 + S_1x + \dots + S_{2t}x^{2t} \in \mathbb{F}_{2^8}[x]$, with $2t = d_e - 1$, associated to the received message $r(x)$ are derived by evaluating the latter at each root of the generator polynomial $g(x)$ of the code, i.e., $S_j = r(\alpha^j) = c(\alpha^j) + e(\alpha^j) = e(\alpha^j)$ for each $j \in \{1, \dots, d_e - 1\}$. The corresponding evaluation circuit, shown in Fig. 2, applies Horner's method for polynomial evaluation [19], processing a symbol of the input polynomial $r(x)$ at each clock cycle, from r_{n_e-1} to r_0 . Each multiplier is fed with a fixed operand thus we applied the multiplier design optimizations discussed before. At the end of processing all n_e symbols of $r(x)$, the j -th memory element, with $1 \leq j \leq n_e - k_e = d_e - 1$, will store the j -th syndrome value, i.e., $r_0 + r_1\alpha^j + \dots + r_{n_e-1}(\alpha^j)^{(n_e-1)} = r(\alpha^j) = S_j$. If all $2t$ syndrome values are zero, then $c(x) = r(x)$ and no errors have occurred. Otherwise, the decoding algorithm attempts to solve the following set of simultaneous equations:

¹In the official HQC specification, the constant term of the gen. polynomial for the RS code employed in hqc128 is reported as 9 instead of 89

$$\begin{cases} Y_1 X_1 + Y_2 X_2 + \dots + Y_{n_e} X_{n_e} = S_0 \\ Y_1 X_1^2 + Y_2 X_2^2 + \dots + Y_{n_e} X_{n_e}^2 = S_1 \\ \vdots \\ Y_1 X_1^{2^t} + Y_2 X_2^{2^t} + \dots + Y_{n_e} X_{n_e}^{2^t} = S_{2^t-1} \end{cases} \quad (1)$$

where each $X_1, X_2, \dots, X_{n_e}$ variable indicates the presence of an error in the i -th received symbol, i.e., $X_i \in \{0, 1\}$, while $Y_1, Y_2, \dots, Y_{n_e}$ are the values of the error symbols (assuming that when $X_i = 0$ then also $Y_i = 0$). An alternate strategy to obtain the same information captured by Eq. (1) considers the methods for solving the following key equation, which is based on the definitions of *error locator polynomial*

$$\Lambda(x) = \prod_{i=1}^{n_e} (1 - X_i x) = 1 + \Lambda_1 x + \dots + \Lambda_{n_e} x^{n_e}$$

and *error evaluator polynomial* $\Omega(x) = \sum_{i=1}^{n_e} Y_i X_i \prod_{j=1, j \neq i}^{n_e} (1 - X_j x) = \Omega_0 + \Omega_1 x + \dots + \Omega_{n_e-1} x^{n_e-1}$: $\Omega(x) = \Lambda(x) S(x) \bmod x^{2^t}$. It is well-known that the coefficients of both $\Lambda(x)$ and $\Omega(x)$ can be computed in $O(t^2) \mathbb{F}_2$ s operations through the so called Berlekamp–Massey algorithm [20]. We employed the design of the Enhanced Parallel Inversionless Berlekamp–Massey Algorithm (ePIBMA) introduced in [21] to derive $\Lambda(x)$ and benefited from the small size of the finite field required for the RS code employed by HQC to realize the finite field inversions necessary to compute $\Omega(x)$, via a 2 KiB read-only memory. The ePIBMA architecture achieves a high clock frequency, since its critical path contains only one multiplication and one addition, and requires only $4t + 2$ multipliers and registers.

2) *Reed-Muller Encoder and Decoder*: The HQC specification describes a binary 1-st order Reed-Muller (RM) code with parameters $[n_i = 2^l, k_i = l, d_i = 2^{l-1}] = [128, 8, 64]$, with $l=7$, extended with a repetition code having each codeword composed by m replicas of the same RM original codeword. For the NIST security level 1, the extended RM code $[384, 8, 192]$ is obtained with $m=3$; for the security levels 3 and 5, the extended RM code $[640, 8, 320]$ is obtained with $m=5$.

a) *Reed-Muller Encoder*: To derive the codewords corresponding to each 8-bit input message, our design considers the RM generation matrix specified by the authors of HQC and follows the traditional vector-matrix multiplication strategy. A straightforward realization would employ $k_i = 8$ multiplexers with 128 input bits, and $k_i - 1 = 7$ xor gates 128-bits wide. A simple, yet effective optimization, we applied in our design of the RM encoder considers each row in the (fixed) generation matrix as a sequence of 32-bits words and observes the presence during the multiplication computations yielding the same intermediate values. As a consequence, we realize the vector-matrix multiplication with only 9 multiplexers, with 32 input bits, and 9, 32-bits, banks of xor gates.

b) *Reed-Muller Decoder*: Since the RS decoder acting as the second stage of the RS-RM concatenated decoder needs the entire output of the RM decoder, minimizing the latency of the RM decoder plays a significant role in our

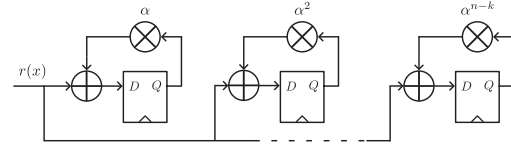


Fig. 2: Syndrome polynomial computation

design. Therefore, in designing the RM decoder we prioritized performance over resource reuse. The extended RM codeword is initially de-duplicated accumulating in a flip-flop-based buffer the bitwise sum of the m -fold 128-bit replicas, via 128 independent adders, to the end of dealing with the plain 1st order RM code $[n_i = 2^l, k_i = l, d_i = 2^{l-1}] = [128, 8, 64]$. Our design of the RM decoder follows the implementation of the Maximum Likelihood (ML) decoder computing a fast Hadamard transform, firstly introduced by [22], which requires only $O(n_i \log(n_i))$ binary operations and $\log(n_i)$ clock cycles in contrast with the $O(n_i^2)$ binary operations and $O(n_i)$ clock cycles provided by a plain ML-based RM decoder. We find the maximum absolute value in the result of the Hadamard transform with a pipelined comparator tree computing pairwise maxima, acting on a tunable-sized input vector.

C. CSPRNG and Hash Functions

In the official specification of HQC.KEM, the Cryptographically Secure Pseudo Random Number Generators (CSPRNGs) and the cryptographic hash functions (HASH-G and HASH-K) that appear in the definition of KEYGEN, ENCAPSULATE and DECAPSULATE algorithms must be based on building blocks that are compliant with the ones detailed in the SHA-3 NIST standard specification [15]. We employ a high performance design for the Keccak [23] *sponge function* [15], computing a mapping between any variable-length bitstring taken as input and a configurable(variable)-length bitstring yielded as output. Our design realizes Keccak $[r, c](\dots)$, with $r=1088$, $c=512$, which has a b -bit wide inner state, with $b=r+c=1600$ bits, and an r -bit auxiliary buffer, both realized as flip-flops. The r parameter defines the size of the blocks into which input and output data streams are partitioned to be processed or yielded, while the c parameter denotes the additional portion of the Keccak state which is never output, providing its security margin. The input data stream is padded so it is segmented into blocks of the same size, while the sequential processing of every input block, a.k.a. *absorption* operation, is carried out computing for each of them a bitwise xor with the first r bits of the internal state, followed by the application of a fixed transformation, denoted as Keccak- $f[b]$, built with xor, and and not Boolean operations, iterated 24 times. The output data stream is obtained one r bit block at a time, copying the first r -bits of the internal state of the sponge function, and applying the Keccak- f transformation on the entire state after its emission if more output is needed. If the length of the output stream is not a multiple of r , the last output block is truncated. The SHA-3 standard defines four hash functions, named as SHA3-224, SHA3-256, SHA3-384, and SHA3-512, depending on

the length of the digest, and two eXtensible Output Functions (XOFs) named as Secure Hash Algorithm Keccak (SHAKE) with either a 128-bit or a 256-bit security level (against all attacks), with configurable output length d . In particular, SHAKE128 and SHAKE256 take a variable-length bitstring $\mathbf{m}$ as input and concatenate it with a fixed string, while yielding a d -bit string as output using the said sponge function as follows: $\text{SHAKE128}(\mathbf{m}, d) = \text{Keccak}[1344, 256](\mathbf{m}||1111)$ and $\text{SHAKE256}(\mathbf{m}, d) = \text{Keccak}[1088, 512](\mathbf{m}||1111)$. The HQC.KEM specification employs only SHAKE256: we designed a hardware component with 64-bit wide input and output data ports, computing the Keccak- f transformation in 24 clock cycles, as the SHAKE module is in the critical scheduling path of all HQC.KEM primitives. Realizing the Keccak- f [1600] transformation acting on the whole internal state memory with a latency of 24 clock cycles provides a design that exhibits the minimum latency with the maximum possible working frequency. Indeed, in [24] the authors realize the Keccak- f function in less than 24 clock cycles, at the cost of reducing the working frequency, while [13] saves hardware resources via sequential processing of the internal state, also resulting in a reduction of the working frequency (in addition to requiring more than 24 clock cycles per Keccak- f computation). The size of the input/output (I/O) ports of the SHAKE256 module is computed considering that an r -bit I/O block must be accessed via a shared buffer in 24 cycles at most (while the Keccak- f is computed) to avoid stalls.

We follow the HQC specification building all $\text{CSPRNG}(\eta, S)$ CSPRNGs expanding a seed with a SHAKE256 module, and employing the output stream as the pseudorandomness source. This fulfills the requirements of drawing pseudorandom bitstrings, such as the dense polynomials. We provide domain separation between the two hash functions HASH-G and HASH-K (realized as SHA-3), from the realization of the CSPRNG as a seed expander, appending one-byte constants 0x02, 0x03, 0x04, respectively, to the original input message to each primitive.

a) *Sparse Polynomial Sampler*: Besides sampling polynomials uniformly from $\mathbf{R}$, HQC require to sample two or more $(p-1)$ -degree polynomials from $\mathbf{R}$, each with a fixed number of non-null coefficients (i.e., $\text{CSPRNG}(\{0, 1\}^{320}, \mathbf{R}_w \times \mathbf{R}_w)$, and $\text{CSPRNG}(\{0, 1\}^{512}, \mathbf{R}_{w_e} \times \mathbf{R}_{w_r} \times \mathbf{R}_{w_r})$). To this end, the constant-time algorithm appeared in [25], has been included in the official specification of HQC starting from fourth revision. We note that changing this sampling approach will lead to non interoperable HQC implementations, as the pseudorandom sampling is repeated by both the encapsulation and decapsulation module. Furthermore, in [26] the authors proved that the a straightforward, non-constant time rejection sampling approach allows to observe timing variations dependent on the secret key in the deterministic re-encryption executed in the HQC.KEM-DECAPSULATE primitive, leading to a successful key recovery attack based on such a side channel. Thus, previous HQC implementations adopting a rejection-sampling based approach for the generation of sparse

polynomials, as the HLS HQC implementation presented in [12], do not provide timing-side channel immunity. From an hardware design perspective, the notable aspects of the sampling algorithm for sparse polynomials with Hamming weight w , described in [25], are the use of an exact amount of randomness, $32 \cdot w$ bits, and the requirement to perform modulo operations between a 32-bit dividend and a 16-bit divisor, for which we used a *shift-and-subtract* algorithm. In our design, we realized a pipelined divisor, aiming at reducing the 16 clock cycles latency a straightforward design, by realizing a sequence of stages, where each of them can be configured to merge multiple iterations of the *shift-and-subtract* algorithm. We evaluated the maximum working frequency of the design, while varying the number of merged iterations, and obtained that, for the divisor not to be the element lowering the working frequency of the overall design, we need to have each stage compute a single iteration of the *shift-and-subtract* algorithm.

D. Schedule of Operations

To provide a hardware design aimed at a final ASIC deployment, we limited the number of memories to an industrially reasonable quantity of 5 trading-off some performance advantages coming from the concurrent execution of some operations (e.g., polynomial multiplications). Furthermore, each memory port is exclusively bound to a single memory module during the execution of a specific state of the global FSA, using the simplest arbitration logic. We opted for a flattened HDL module hierarchy, including the components of the HQC.PKE modules in the ones implementing the HQC.KEM, benefiting from the reduced latency provided by the manually routed crossbar interconnecting the computing modules to both memories and the SHAKE256 component. Given the constraints noted above, the schedule of the operations executed by each top-level module resulted to be the same, regardless of the distinct parameters reported in the official documentation of HQC for the NIST security levels 1, 3, and 5, denoted as hqc128, hqc192, hqc256, respectively.

1) *HQC.KEM Key Generation Module*: The HQC.KEM-KEYGEN schedule, reported in Figure 3c, starts by expanding the seed φ into the dense public polynomial h (step K_0 and K_1), and expanding the seed σ into the two private polynomials x and y (step K_2 and K_3). Devising the scheduling of the HQC.KEM-KEYGEN, allowed us to propose a variation to the HQC specification to speed up the algorithm, at no resource cost or security loss. Since the sparse polynomial y can be employed before the polynomial x , and may need to be stored by the decapsulation module, if present, it makes sense to sample it as the first element without taking time discarding the first $w \cdot 32$ -bits linked to x squeezed from the SHAKE module, as per the HQC specification. The HQC.KEM-KEYGEN schedule goes on by sampling the sparse polynomial y in step K_3 , and multiplying it by h in step K_4 , while x is being sampled. Finally in step K_5 we construct the syndrome s adding the sparse polynomial x to the result of the multiplication. The sparse polynomials x and y are sampled sequentially with

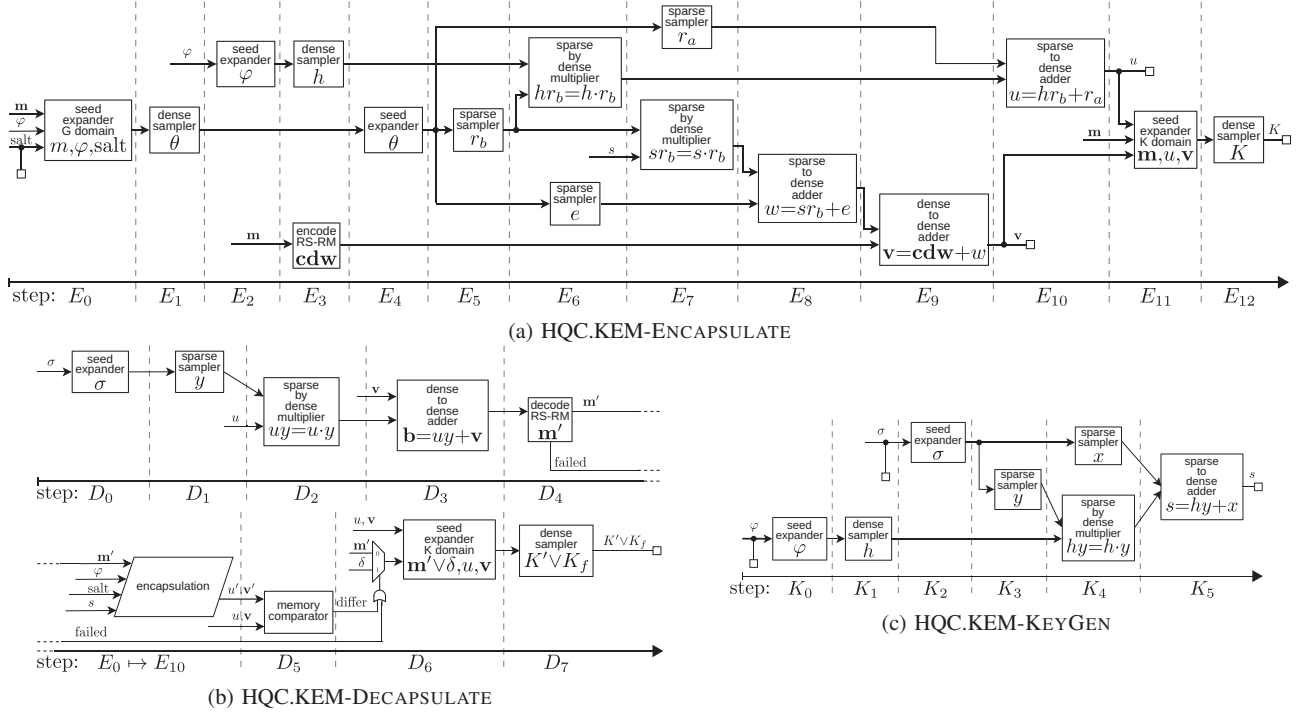


Fig. 3: Operation schedules for HQC.KEM scheme. A white box represents an output value

h , as replicating the required SHAKE module would raise significantly the hardware resource requirements.

2) *HQC.KEM Encapsulation Module*: The module starts by absorbing with SHAKE the message $\mathbf{m}$, the salt and the public key seed φ (step E_0), squeezing out the intermediate seed θ (step E_1). Subsequently, while the parity-check polynomial h is expanded from φ , the message $\mathbf{m}$ is encoded by the RS-RM encoder (steps E_2 and E_3). After absorbing the encryption salt θ in the seed-expander SHAKE domain, the sparse polynomials r_a , r_b , and e are sampled. Scheduling these operations, we propose a further optimization of the HQC specification, in the same line of thought as the one in HQC.KEM-KEYGEN. We sample the values r_b , e , and r_a in this order, allowing the parallel execution of two sparse-to-dense multiplications during the first two sampling actions (steps E_5 to E_7). The sparse polynomial e is added to result of the multiplication of s by r_b in step E_8 by flipping the binary coefficients at the index locations specified by e , and the result is added to the codeword obtaining $\mathbf{v}$. Finally, the addition of the sparse polynomial r_a to the result of the multiplication of h by r_b is carried out producing the polynomial u (step E_{10}). After absorbing the message $\mathbf{m}$ and the ciphertext $u\|\mathbf{v}$ in the $\mathcal{K}$ domain (step E_{11}), the first 512-bits of SHAKE's internal state will contain the session key K , stored then in memory.

3) *HQC.KEM Decapsulation Module*: The scheduling of this module starts benefiting from the optimization of the HQC.KEM-KEYGEN procedure, as it is possible to sample the sparse polynomial y (step D_0) without discarding any output from the SHAKE based CSPRNG, as it would have been necessary if the x polynomial were sampled first. The sampled

sparse polynomial is multiplied by the first portion of the ciphertext u (step D_2) and added to the second portion $\mathbf{v}$ (step D_3). After step D_3 , the result is a decodable message $\mathbf{m}'$, fed to the concatenated RS-RM decoder in step D_4 , concluding the HQC.PKE decryption. Due to the Hofheinz-Hövelmanns-Kiltz framework, the steps from E_0 to E_{10} repeat the encapsulation procedure using the decoded message $\mathbf{m}'$, obtaining $u'\|\mathbf{v}'$: if these values do not match received ciphertext $u\|\mathbf{v}$, or the decoding of the message was not successful, the secret value δ from HQC's private key is employed to derive a fake session key through SHAKE, instead of $\mathbf{m}'$. The session key is generated absorbing the ciphertext $u\|\mathbf{v}$ (step D_6), and storing the first 512-bits of SHAKE's internal state (step D_7).

IV. EXPERIMENTAL EVALUATION

We realized our designs as register transfer level (RTL) descriptions using the SystemVerilog language (sources available at DOI:10.5281/zenodo.10135379), parametrizing our code to explore different values for the memory word size B . All modules were validated via a behavioral simulation using the Python language with the `cocotb` framework [27] and following the IEEE 1800.2 Universal Verification Methodology (UVM) [28]. During the development, we targeted the Artix-7 FPGA line; in particular, we employed the xc7a200tfbg484-3 target. We validated it on a physical board (Digilent's Arty A7-100T), with the smaller xc7a100tcsbg324-1 chip, as our designs still fit it, employing the Known Answer Tests (KAT) distributed with the latest HQC specification, and communicating via a UART module. Willing to compare fairly our results with other

TABLE II: Performance of polynomial samplers (memory block size $B = 128$ bits). Area-Time product in eSlices $\cdot \mu s$.
* Contribution of DSP units not present in the AT product

Param. set	Poly format	Work	Resources				freq. MHz	Latency		AT prod.	
			LUT	FF	BRAM	DSP		eSlice	μs		
hqc128	dense	our	86	330	1.0	0	150	359	592	1.65	0.25
			1055	1189	2.0	0	520	230	2335	10.15	5.28
	sparse	[13] CWW	201	229	1.0	4	179	201	3062	15.23	2.73*
		[13] FNB	316	124	2.0	0	335	223	1479	6.63	2.22
		[11]	1560	766	2.0	0	646	170	976	5.74	3.71
			89	333	1.0	0	151	347	1160	3.34	0.50
hqc192	dense	our	1010	1178	2.0	0	509	237	5207	21.97	11.18
			211	245	1.0	5	181	200	6817	34.08	6.17*
	sparse	[13] CWW	295	125	2.0	0	330	236	2226	9.43	3.11
		[13] FNB									
		[11]	1553	761	3.0	0	773	185	1636	8.84	6.84
hqc256	dense	our	89	333	1.0	0	151	357	1840	5.15	0.78
			1027	1172	2.0	0	513	225	8834	39.26	20.14
	sparse	[13] CWW	216	248	1.0	5	182	204	11487	56.31	10.25*
		[13] FNB	314	192	2.5	0	399	242	3248	13.42	5.36
		[11]	1569	779	3.0	0	777	181	2268	12.53	9.74

works, we introduce an *equivalent slices* (eSlice) synthetic indicator to summarize the use of lookup tables (LUT), flip-flops (FF), and block memories (BRAM) specific to a chosen target. Considering that a slice in a Xilinx 7-Series FPGA includes four 6-input LUTs and eight FFs, and that each BRAM36 contains up to 32 Kib of data, we derive the number of eSlices as $\max\{\lceil \text{LUT}/4 \rceil + \text{BRAM} \times 128, \lceil \text{FF}/8 \rceil\}$. The conversion of BRAMs in LUTs is achieved considering only the data storage without the extra logic incurred for each read or write port, since each LUT can contain 2^6 bits. While we acknowledge that this metric does not match exactly the number of slices the design occupies after the implementation phase, we believe that it allows a fair comparison of different FPGA-tailored designs considering the heterogeneous resources present in modern devices. As for the efficiency metric, we employ the usual Area-Time (AT) product, computed as the number of eSlices times the latency in either microseconds or nanoseconds for readability's sake.

a) *Polynomial Samplers*: In Table II, we report the area and performance results for our polynomial samplers, and compared with other works. The dense polynomial sampler performance is bounded by the one of the SHAKE256 module, which limits the sampling of polynomials for values of B greater than 64. The 16-stage fully-pipelined modulo operator does not limit the working frequency, while taking 848 LUTs and 808 FFs, which is roughly 80% of the resources of the sparse polynomial sampler. Our design compares favourably to the Constant Weight Word (CWW) solution of [13], which implements the same sampling algorithm but performs the modulo operation with a generic Barrett reduction requiring 4 to 5 Digital Signal Processing (DSP) units. Despite employing dedicated resources, their realization achieves a lower working frequency, and is 50% slower than our design. Comparing our solution with other constant-time implementations based on different algorithms from the one in the official HQC specification, the Fast and Non-Biased (FNB) variant in [13] achieves comparable frequencies with less hardware resources; the implementation in [11] achieves failure-free constant time execution, although with better performance than ours.

TABLE III: Performance of hqc256 polynomial multipliers. Area-Time (AT) product in eSlices $\cdot ns$

Block size	Read ports	Shift stages	Resources				freq. MHz	Latency		AT prod.
			LUT	FF	BRAM	eSlice		CC	μs	
32	1	1	403	405	4.5	677	248	236723	954	646
		2	466	411	4.5	693	249	236724	950	658
		3	483	448	4.5	697	243	236725	974	679
		4	487	487	4.5	698	262	236726	903	630
	4	1	1127	803	7.0	1178	210	59637	283	334
		2	1341	922	7.0	1232	238	59638	250	308
		3	1417	1065	7.0	1251	234	59639	254	314
		4	1408	1205	7.0	1248	232	59640	257	322
	1	1	696	655	4.5	750	227	118692	522	392
		2	848	665	4.5	788	251	118693	472	372
		3	828	744	4.5	783	267	118694	444	348
		4	825	808	4.5	783	261	118695	454	356
64	1	1	2615	1306	7.0	1550	183	29904	163	253
		2	2566	1529	7.0	1538	237	29905	126	194
		3	2538	1829	7.0	1531	222	29906	134	206
		4	2552	2099	7.0	1534	250	29907	119	183
	4	1	963	1161	6.5	1073	173	59742	345	370
		2	1435	1173	6.5	1191	211	59743	283	337
		3	1442	1315	6.5	1193	254	59744	235	280
		4	1572	1464	6.5	1225	258	59745	231	283
128	1	1	4962	2333	11.0	2649	155	15054	97	257
		2	4779	2747	11.0	2603	194	15055	77	202
		3	4544	3304	11.0	2544	214	15056	70	178
		4	5101	3903	11.0	2684	233	15057	64	173

b) *Polynomial Adders*: Our design space exploration for the polynomial adders confirmed the expectation that dense adders increase their efficiency when the value of B grows; by contrast, sparse adders yield a constant efficiency. We also note that, with $B=128$, the dense polynomial adder requires less cycles than the sparse one. Our approach yields polynomial adders taking ≤ 164 LUTs and ≤ 313 FFs for all HQC security parameters, attaining frequencies ≥ 499 MHz. Comparing our approach with the one of [13], we employ a higher number of FFs, as this allows us to boost the maximum achievable frequency by $1.5 \times$ to $2 \times$. We note that the latencies in [13] suggest the use of pipelining in the sparse polynomial adder, leading to results affected by rare computation errors due to read-after-write dependencies.

c) *Polynomial Multipliers*: Table III reports for hqc256 the results of our parameter exploration for the polynomial multipliers varying: the memory word size $B \in \{32, 64, 128\}$, the number of read ports to the memories containing the dense operand (1 or 4), and the number of pipeline stages for each Barrel shifter (1 to 4) – results for hqc128, hqc192 follow similar trends. The demands on the logic resources scale linearly with the number of read ports, due to the replication of the independent logic rotating the received data.

Increasing the number of stages in the pipelined Barrel shifter, we obtain substantial improvements in the maximum frequency achieved for the designs using either memory words as large as possible or more read ports. We note that the AT product worsens for simpler designs, e.g., the ones with $B=32$.

The number of FFs required by the input buffers (each realized as a FIFO queue with depth 3) ranges between $1/2$ and $5/6$ of the total number of FFs employed for the whole multiplier. Using four read ports improves the latency by $\approx 4 \times$ and substantially increases the efficiency, at the cost of a small frequency drop. An extra pipeline stage would further reduce the penalty in frequency, although analyzing the overall

TABLE IV: Performance of RS-RM en- and decoder. RM dec. compares 16 elements at a time. AT product in eSlices · ns

Module	Code	Param. set	Design	LUT	FF	BRAM	eSlice	freq. MHz	Latency CC	μs	AT prod.
encoders	RM	hqc128	our	246	322	2.0	318	434	194	0.45	0.14
		hqc192	our	247	329	2.0	318	396	336	0.85	0.27
		hqc256	our	255	329	2.0	320	411	506	1.23	0.39
	RS	hqc128	our	299	299	0.5	139	419	52	0.12	0.02
		hqc192	our	317	316	0.5	144	397	62	0.16	0.02
		hqc256	our	527	526	0.5	196	396	96	0.24	0.05
	RS-RM	hqc128	our	532	620	2.0	389	374	148	0.40	0.15
		hqc192	our	858	922	2.0	471	270	97	0.36	0.17
		hqc256	our	2019	603	0.0	505	-	7244	-	-
	RM	hqc128	[13]	570	644	2.0	399	394	290	0.74	0.29
		hqc192	[13]	1011	1088	2.0	509	298	131	0.44	0.22
		hqc256	[13]	743	854	2.0	442	386	460	1.19	0.53
decoders	RM	hqc128	our	2761	1642	1.0	819	219	1244	5.68	4.65
		hqc192	our	3606	1850	1.0	1030	193	1626	8.42	8.68
		hqc256	our	3599	1835	1.0	1028	217	2612	12.04	12.37
	RS	hqc128	our	3184	1250	0.5	860	273	97	0.36	0.31
		hqc192	our	3512	1378	0.5	942	265	117	0.44	0.42
		hqc256	our	6009	2309	0.5	1567	249	185	0.74	1.16
	RS-RM	hqc128	our	5896	3364	2.5	1794	212	1293	6.10	10.94
		hqc192	our	2817	3779	2.5	1025	205	4611	22.49	23.06
		hqc256	our	10154	2569	3.0	2923	-	68619	-	-
	RM	hqc128	[13]	7219	3561	2.5	2125	219	1685	7.69	16.35
		hqc192	[13]	3257	4727	2.5	1135	212	5485	25.87	29.37
		hqc256	[13]	10090	4472	2.5	2843	225	2705	12.02	34.18
	RS	hqc128	[13]	3679	5574	2.5	1240	206	9199	44.66	55.37

latencies in the top-level designs, we deemed it unnecessary.

d) *RM-RS Encoder and Decoder*: The RM-RS encoders proved remarkably compact, fast, and efficient. As depicted in Table IV, their concatenation produced a slower design with respect to [13]. However, we did not apply any further optimizations since the latency of the module is fully hidden by the dense polynomial sampler, which is run in parallel in the operation schedule. For the Reed-Muller decoder, we explored the speed/area trade-off given by the parallel comparison of multiple elements. As the comparison tree is fully pipelined, the number of FFs used quickly rises with the size of comparison block: we obtain the best efficiency with 16 parallel comparisons, with a comparatively small resource increment. For the Reed-Solomon decoder, the *ePIBMA* module contributes for the vast majority to the logic resource usage ($\approx 75\%$ of LUTs) due to the large amount of general $F(2^8)$ multipliers required. Comparing the concatenated decoder with [13], our design uses more ($2\times$ to $2.74\times$) LUTs, while requiring less FFs; however, our solution from $3.36\times$ to $3.71\times$ faster and about twice more efficient in the AT metric.

e) *Full HQC.KEM*: The HQC.KEM modules interact with true dual-port RAMs, mapped to Block RAMs (BRAMs), providing two 128-bits wide ports configurable to perform either a read or a write operation per port. For modules accessing the memory with a smaller word size, we used a bus adapter translating addresses and performing the sub-word selection. We recall that we limited the number of memories used to 5 in any top-level design, to provide plausible figures for a final ASIC deployment. The maximum memory space is required by the decapsulation procedure, which uses 22.28, 44.73, and 71.48 KiB when targeting the hqc128, hqc192, and hqc256 parameters set, respectively. Table V provides a comparison of our design with the current ones in the state of

TABLE V: Performance of HQC top-modules w/o SHAKE256 (5520 LUTs and 2810 FFs). AT product in eSlices · ns

Module	Param. set	Design	LUT	FF	BRAM	eSlice	freq. MHz	Latency CC	μs	AT prod.
keygen	hqc128	[13]	2396	901	10.0	1879	179	15759	88	165
		[12] (HLS, perf.)	8324	5273	6.0	2849	150	40427	270	768
		our	6827	5444	20.0	4267	208	6203	30	127
	hqc192	[13]	2342	926	10.0	1866	189	42106	222	415
		our	7150	5462	20.0	4348	207	14940	72	314
		[13]	2342	942	10.0	1866	188	82331	437	817
	hqc256	our	6845	5457	20.0	4272	201	27825	138	591
		[13] (balanced)	4145	2128	13.0	2701	179	33438	186	504
		[13] (high speed)	5828	2531	15.0	3377	179	22423	125	423
	hqc128	[12] (HLS, perf.)	13179	9851	10.0	4575	152	89110	586	2682
		our	7064	6378	20.0	4326	168	13314	79	343
		[13] (balanced)	4278	2412	15.0	2990	182	90346	496	1484
encap.	hqc192	[13] (high speed)	6434	2821	17.0	3785	196	57314	292	1106
		our	7629	6427	20.0	4468	175	31460	180	803
		[13] (balanced)	4809	3041	15.0	3123	182	177154	973	3039
	hqc256	[13] (high speed)	6898	3417	17.0	3901	196	108527	553	2160
		our	7406	6631	20.0	4412	187	58588	313	1382
		[13] (balanced)	8984	6596	20.0	4806	192	48212	251	1206
	hqc128	[13] (high speed)	10959	7003	22.0	5556	179	37192	207	1154
		[12] (HLS, perf.)	15304	11704	18.0	6130	152	193082	1270	7787
		our	13327	9488	20.0	5956	167	19880	119	709
	hqc192	[13] (balanced)	9969	7911	22.0	5309	186	125805	676	3590
		[13] (high speed)	11915	8320	24.0	6051	186	92773	498	3018
		our	15982	9878	24.0	7068	161	46160	287	2026
	hqc256	[13] (balanced)	10931	9424	22.0	5549	186	248338	1335	7408
		[13] (high speed)	12868	9794	24.0	6289	186	179711	966	6076
		our	18310	10989	27.5	8098	151	86028	570	4614

the art; we use the same convention of [13], and extract the resources required by the SHAKE module. While our high efficiency design for HQC.KEM keygen employs more hardware resources than [13], we are able to improve the efficiency, as shown by the AT product, thanks to a $2.93\times$ improvement in the computation time. When compared with the official HLS implementation [12] we use $\approx 20\%$ less LUTs and a similar number of FFs. The considerable savings on the block RAMs in [12] are likely due to the high count of distributed memories they used. The maximum frequency attained in [12] is $1.4\times$ lower in turn giving our solution a $6.05\times$ better area efficiency. For the encapsulation and decapsulation modules, the difference in the resource usage decreases when compared with the fastest RTL implementation of [13]. At the same time, our performance improvements remain in the $1.58\times$ to $1.76\times$ range. Similar results are obtained for the efficiency gain, which ranges from $1.23\times$ to $1.62\times$. Comparing with the HLS implementation of [12], we used less eSlices, and completed the encapsulation and decapsulation $7.41\times$ and $10.67\times$ faster, respectively. Our solution improves the efficiency of [12] by $7.82\times$ for the encapsulation, and $10.98\times$ for the decapsulation. Changing the sequence of sparse polynomial sampling (i.e., y before x during the key generation, or only x during the decapsulation, and r_b, e before r_a during the encapsulation) made us save from 13% (hqc256) to 32% (hqc128) of latency w.r.t. an implementation abiding to the HQC specification.

V. CONCLUDING REMARKS

We presented a full HQC.KEM RTL design, improving on the current state of the art in both latency and efficiency. Our design achieves encapsulation and decapsulation each in less than $120\mu s$ for hqc128 on a budget Xilinx Artix A7-100T FPGA.

REFERENCES

- [1] C. Chen et. al., "NTRU: A submission to the NIST post-quantum standardization effort," 2022. [Online]. Available: <https://ntru.org/>
- [2] F. Antognazza, A. Barenghi, G. Pelosi, and R. Susella, "A Flexible ASIC-Oriented Design for a Full NTRU Accelerator," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference, ASPDAC 2023, Tokyo, Japan, January 16-19, 2023*, A. Takahashi, Ed. ACM, 2023, pp. 591–597. [Online]. Available: <https://doi.org/10.1145/3566097.3567916>
- [3] S. Schmieg, "PQC at Google," Invited talk at The 14th International Conference on Post-Quantum Cryptography, PQCrypto 2023 <https://pqcrypto2023.umi.acs.io/slides/Invited.3.pdf>
- [4] ISE Crypto PQC working group, "Securing tomorrow today: Why Google now protects its internal communications from quantum threats," <https://cloud.google.com/blog/products/identity-security/why-google-now-uses-post-quantum-cryptography-for-internal-comms>
- [5] V. Lyubashevsky, L. Ducas, E. Kiltz, T. Lepoint, P. Schwabe, G. Seiler, D. Stehle, and S. Bai, "Kyber Supporting Documentation," [Online]. Available: <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>, (Last accessed on 4 Sept. 2023), 2021.
- [6] D. J. Bernstein, T. Chou, C. Cid, J. Gilcher, T. Lange, V. Maram, I. von Maurich, R. Misoczki, R. Niederhagen, E. Persichetti, C. Peters, N. Sendrier, J. Szefer, C. J. Tjhai, M. Tomlinson, and W. Wang, "Classic McEliece Supporting Documentation," [Online]. Available: <https://classic.mceliece.org/mceliece-spec-20221023.pdf>, (Last accessed on 4 Sept. 2023), 2022.
- [7] N. Aragon, P. Barreto, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, S. Gueon, T. Guney, C. A. Melchor, R. Misoczki, E. Persichetti, N. Sendrier, J.-P. Tillich, G. Zemor, V. Vasseur, S. Ghosh, and J. Richter-Brokmann, "BIKE Supporting Documentation," [Online]. Available: https://bikesuite.org/files/v5.0/BIKE_Spec.2022.10.10.1.pdf, (Last accessed on 4 Sept. 2023), 2022.
- [8] C. A. Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zemor, J. Bos, A. Dion, J. Lacan, J.-M. Robert, and P. V. and, "HQC Supporting Documentation," [Online]. Available: http://pqc-hqc.org/doc/hqc-specification_2023-04-30.pdf, (Last accessed on 4 Sept. 2023), 2023.
- [9] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A Modular Analysis of the Fujisaki-Okamoto Transformation," in *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, ser. Lecture Notes in Computer Science, Y. Kalai and L. Reyzin, Eds., vol. 10677. Springer, 2017, pp. 341–371. [Online]. Available: https://doi.org/10.1007/978-3-319-70500-2_12
- [10] Y. Tu, P. He, Ç. K. Koç, and J. Xie, "LEAP: Lightweight and Efficient Accelerator for Sparse Polynomial Multiplication of HQC," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 31, no. 6, pp. 892–896, 2023. [Online]. Available: <https://doi.org/10.1109/TVLSI.2023.3246923>
- [11] P. He, Y. Tu, and J. Xie, "LOCS: LOW-Latency and ConStant-Timing Implementation of Fixed-Weight Sampler for HQC," *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260004183>
- [12] C. A. Melchor, J. Deneuville, A. Dion, J. Howe, R. Malmain, V. Migliore, M. Nawan, and K. Nawaz, "Towards Automating Cryptographic Hardware Implementations: A Case Study of HQC," in *Code-Based Cryptography - 10th International Workshop, CBCrypto 2022, Trondheim, Norway, May 29-30, 2022, Revised Selected Papers*, ser. Lecture Notes in Computer Science, J. Deneuville, Ed., vol. 13839. Springer, 2022, pp. 62–76. [Online]. Available: https://doi.org/10.1007/978-3-031-29689-5_4
- [13] S. Deshpande, M. Nawan, K. Nawaz, J. Szefer, and C. Xu, "Fast and Efficient Hardware Implementation of HQC," *IACR Cryptol. ePrint Arch.*, 2023. [Online]. Available: <https://eprint.iacr.org/2022/1183>
- [14] M. Schöffel, J. Feldmann, and N. Wehn, "Code-based Cryptography in IoT: A HW/SW Co-Design of HQC," *CoRR*, vol. abs/2301.04888, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2301.04888>
- [15] M. Dworkin, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," Federal Inf. Process. Stds. (NIST FIPS), National Institute of Standards and Technology, Gaithersburg, MD, United States, [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.202>, (Last accessed on 4 Sept. 2023), 2015.
- [16] S. Lin and D. J. Costello Jr., *Error control coding - fundamentals and applications*, ser. Prentice Hall computer applications in electrical engineering series. Prentice Hall, 1983.
- [17] E. D. Mastrovito, "VLSI Designs for Multiplication over Finite Fields $GF(2^m)$," in *Applied Algebra, Algebraic Algorithms and Error-Correcting Codes, 6th International Conference, AAECC-6, Rome, Italy, July 4-8, 1988, Proceedings*, ser. Lecture Notes in Computer Science, T. Mora, Ed., vol. 357. Springer, 1988, pp. 297–309. [Online]. Available: https://doi.org/10.1007/3-540-51083-4_67
- [18] H. Fan and M. A. Hasan, "A survey of some recent bit-parallel $GF(2^m)$ multipliers," *Finite Fields Their Appl.*, vol. 32, pp. 5–43, 2015. [Online]. Available: <https://doi.org/10.1016/j.ffa.2014.10.008>
- [19] V. Y. Pan, "Methods of Computing Values of Polynomials," *Russian Mathematical Surveys*, vol. 21, pp. 105 – 136, 1966. [Online]. Available: <https://api.semanticscholar.org/CorpusID:250869179>
- [20] J. Massey, "Shift-register synthesis and BCH decoding," *IEEE Transactions on Information Theory*, vol. 15, no. 1, pp. 122–127, 1969.
- [21] Y. Wu, "New Scalable Decoder Architectures for Reed-Solomon Codes," *IEEE Trans. Commun.*, vol. 63, no. 8, pp. 2741–2761, 2015. [Online]. Available: <https://doi.org/10.1109/TCOMM.2015.2445759>
- [22] Y. Be'ery and J. Snyders, "Optimal soft decision block decoders based on fast Hadamard transform," *IEEE Trans. Inf. Theory*, vol. 32, no. 3, pp. 355–364, 1986. [Online]. Available: <https://doi.org/10.1109/TIT.1986.1057189>
- [23] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, "The Keccak reference," [Online]. Available: <https://keccak.team/files/Keccak-reference-3.0.pdf>, (Last accessed on 4 Sept. 2023), 2011.
- [24] G. Bertoni, J. Daemen, M. Peeters, G. V. Assche, and R. V. Keer, "Keccak implementation overview," [Online]. Available: <https://keccak.team/files/Keccak-implementation-3.2.pdf>, (Last accessed on 4 Sept. 2023), 2012.
- [25] N. Sendrier, "Secure Sampling of Constant-Weight Words - Application to BIKE," *IACR Cryptol. ePrint Arch.*, p. 1631, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1631>
- [26] Q. Guo, C. Hlauschek, T. Johansson, N. Lahr, A. Nilsson, and R. L. Schröder, "Don't Reject This: Key-Recovery Timing Attacks Due to Rejection-Sampling in HQC and BIKE," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 3, pp. 223–263, 2022. [Online]. Available: <https://doi.org/10.46586/tches.v2022.i3.223-263>
- [27] FOSSI Foundation, "cocotb - coroutine-based cosimulation testbench environment for verifying VHDL and SystemVerilog RTL using Python," [Online]. Available: <https://www.cocotb.org>, (Last accessed on 4 Sept. 2023), 2019.
- [28] R. Salemi and Siemens EDA, "PyUVM - Universal Verification Methodology based on the IEEE 1800.2 specification implemented in Python," [Online]. Available: <https://github.com/pyuvm/pyuvm>, (Last accessed on 4 Sept. 2023), 2018.